

Tema 2

Mihai Cristian-Laurentiu 2B4, Stamate Valentin 2B4

23.11.2020

Problema 1

a. Considerăm pentru următoarea demonstrație relația $|S| \leq |T|$ deoarece cuplajul satisface întotdeauna clasa cu cardinalul cel mai mic din bipartitie.

Presupunem că există un cuplaj M care saturează toate nodurile din S , și demonstrăm că se satisface condiția lui Hall $\forall A \subseteq S$. Fie $M(A)$ mulțimea tuturor varfurilor din T satisfăcute de cuplajul M , pe o mulțime A fixată. Din definiția cuplajului $|M(A)| = |A|$, dar $M(A) \subseteq N_G(A)$ deoarece toate varfurile din $M(A)$ sunt vecini cu varfurile din A . Astfel $|N_G(A)| \geq |M(A)|$, deci $N_G(A) \geq |A|$. Deci există un cuplaj care satisface o clasă a bipartitiei

b. În continuare vom încerca să explicăm o strategie pentru aflarea celei de-a șasea cărți. Fie cei doi prieteni, Dealerul (cel care va face trucul), și Jucatorul (cel care va "ghici" cartea). Vom folosi doar 5 din cele 6 cărți, astfel că de cincea carte plasată nu va fi necesară pentru aflarea celei ascunse. Dealerul alege cartile pe care le va arăta, cât și ordinea lor. Există C_5^4 modalități de a alege 4 din 5 cărți și $4!$ moduri de a ordona cele 4 cărți (primele patru). Jucatorul poate decoda relativ ușor 24 de combinații dar mai greu cele 120. O metodă pentru a decoda mai ușor mai mult de 24 de combinații este să observe, folosind principiul lui Dirichlet, că în cele 5 cărți pe care Dealerul le folosește trebuie să existe cel puțin două de același semn. Prima carte pe care Dealerul o va arăta este una din acea pereche de două, iar a doua (pe care nu o va arăta) este cartea care urmează să fi ghicită. Presupunem că ordonăm cartile crescător de la $A, 2, 3, \dots, K$ și le plasăm într-o ordine circulară (astfel după K urmează $1, 2, \dots$) și definim distanța între cartea a și cartea b , $\text{distanța}(a, b)$ ca fiind distanța pe cerc de la a la b . Astfel pe orice două cărți a, b $|\text{distanța}(a, b)| \leq 6$. Dacă distanța ar fi 7 sau mai mare atunci ar trebui să existe cel puțin 14 cărți în cerc. De exemplu, fie cartile 4 și Dama (D), Dealerul va arăta ca prima carte Dama pentru că $\text{distanța}(D, 4) = 5$. Apoi Dealerul alege o ordonare a celorlalte 3 cărți (a patra nefiind necesară) pentru a codifica un număr de la 1 la 6. Jucatorul decodifică acest număr și îl adună primei cărți pentru a afla cartea ascunsă. Există multe moduri de a identifica un număr de la 1 la 6 din permutarea celor 3 cărți. Vom folosi în ordine de la 1 la 6: (123), (213), (231), (132), (312), (321). Pentru a afla permutarea, poziția celei mai mici cărți ne spune 1, 2, 3. Din ordinea celorlalte două vom ști dacă să adăugăm sau nu 3; dacă sunt în ordine crescătoare (2, 3) vom adăuga 3, în caz contrar nu. Pentru a putea face ordonarea corectă vom presupune că dacă două cărți au aceeași valoare se vor ordona după simbolul cărții, ordinea fiind fixată de cei doi prieteni în prealabil (Frunza, Inimă, Trefla, Romb: ♣, ♦, ♥, ♠).

Exemplu: Presupunem că Dealerul are cartile $3♦, 6♦, 8♠, 10♣$ și $8♥$, și oricare altă (care nu este necesară pentru a afla cartea ascunsă, o vom nota C). Deoarece avem două cărți de Romb, Dealerul va ascunde $6♦$ sau $3♦$. Dealerul ascunde $6♦$ (sau cealaltă de romb, nu este important, distanța dintre cele două este 3) și afișează secvența $3♦, 8♠, 10♣, 8♥, C$ care codifică 3. Permutarea este (231), 2, 3 fiind în ordine nu vom adăuga 3, ca în cazul (321). Din punctul de vedere al Jucatorului, văzând secvența, vede simbolul cărții ascunse (din prima carte) și combinația următoarelor 3 care codifică numărul 3, adăuga 3 primei cărți și află răspunsul corect ($6♦$), acesta fiind numărul primei cărți și simbolul acesteia + numărul dat de permutare.

Problema 2

a. Stim din definitia din curs: G este n -conex daca G/A este conex $\forall A \subseteq V(G) \ |A| < n$. Cum $A = \{x\} | x \text{ server}$, atunci trebuie sa demonstram ca G este 2-conex. Din enuntul problemei stim ca doi clienti nu pot fi conectati unul cu altul si ca nu pot comunica decat cu serverele deoarece clientii nu pot ruta mesaje. Astfel, un client poate comunica cu un server doar daca serverele se afla se afla intr-o componenta conexa. In cazul minimal subgraful indus de multimea serverelor va fi arbore. Atunci adaugand muchiile dupa cum ne spune proprietatea, vom obtine un graf G' care este 2-conex.

Demonstratie: Presupunem prin reducere la absurd ca subgraful indus de multimea serverelor (G') se deconecteaza la eliminarea unui nod x . Insa vecinii lui x din cele doua componente conexe vor fi conectati printr-o muchie deoarece se aflau la distanta 2, astfel vom obtine o contradictie.

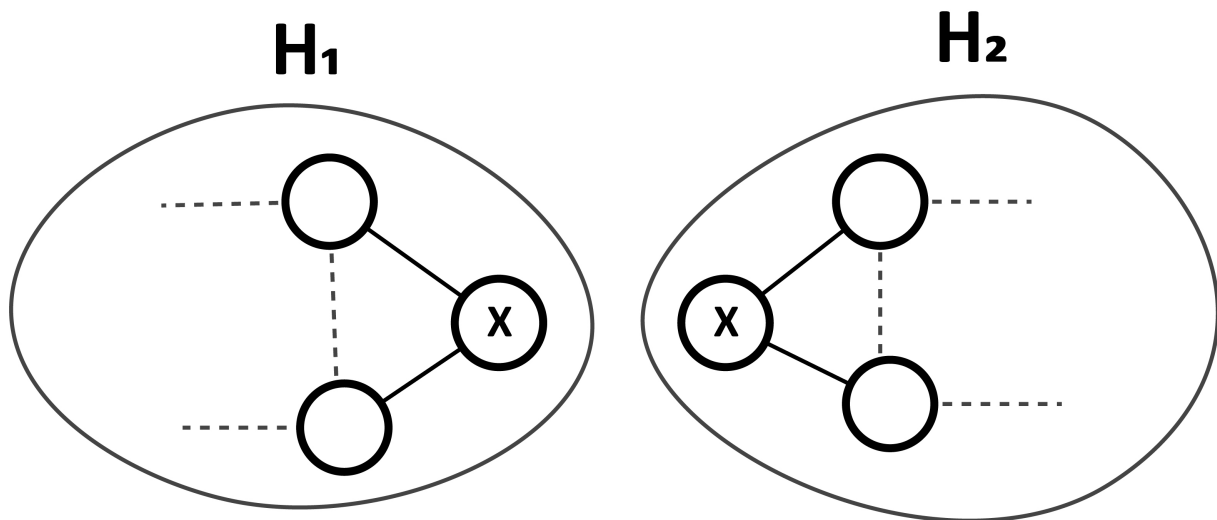
Cum fiecare nod din G' are gradul cel putin 2 atunci la eliminarea unui server toti clientii vor avea gradul cel putin 1 insemnand ca fiecare client va ramane conectat cu cel putin un server si ca poate comunica cu toate celelalte servere, graful G' fiind 2 conex astfel daca un server cade, retea va ramane conexa.

b. Fie G graful asociat retelei, G' (graful dupa modificare), H si H' subgrafuri induse de multimea serverelor in G respectiv G' . Ponind tot de la un caz minimal unde H este un arbore, putem observa urmatorul lucru. Modul cum adaugam muchiile este similar cu modificarile aduse la subpunctul a doar ca nu mai avem restrictia de a trasa o muchie la distanta 2. Astfel noi trebuie sa adaugam doua muchii pentru fiecare doua noduri conectate direct. Aceste muchii se vor crea doua cicluri de lungime minim 2 iar aceste cicluri se vor intersecta in muchia dintre cele doua servere. Astfel fiecare nod avand un vecin dupa modificare vom obtine subgraful H' in care fiecare nod va avea gradul cel putin 3 deoarece pentru un nod oarecare cu grad cel putin 1 va trebui sa adaugam alte doua muchii pentru obtinerea a doua circuite. La fel si pentru nodurile care au deja grad cel putin 2: va trebui sa adaugam o muchie pentru a obtine doua circuite. Daca in graful minimal H ar fi fost mai multe muchii proprietatea nu s-ar fi schimbat deoarece muchiile suplimentare ar fi facut parte din circuitele adaugate anterior. Astfel cum fiecare nod din H' are gradul cel putin 3 putem spune ca este 3-muchie-conex deoarece daca am sterge doua muchii am avea unele noduri cu grad cel putin 1 si s-ar pastra conexitatea in H' . Atunci retea va ramane conectata daca ar cadea doua conexiuni directe dintre servere. Q.E.D.

Problema 3

a. " $\Rightarrow$ " Fie H_1, H_2 doua b-subgrafuri, iar H un subgraf astfel incat $H = (H_1 \cap H_2)$. H contine strict pe H_1 (sau H_2) deci din proprietatea din enunt acesta nu mai este 2-conex, astfel prin eliminarea nodului in care H_1 intersecteaza H_2 subgraful H se deconecteaza, deci nodul de intersectie este punct de articulatie.

" $\Leftarrow$ " Fie x punct de articulatie in G . Stim ca $G \setminus x = G'$ are doua componente conexe H_1 si H_2 iar $|V(b - \text{subgraf})| \geq 2$. Astfel $H_1 \cup x$ este conex dar si $H_2 \cup x$ este conex. Deoarece un b-subgraf are minim doua noduri, pornind de la x putem construi un b-subgraf $= B_1$ in H_1 ($B_1 \in G$) care nu are noduri in H_2 (exceptie nodul comun) deoarece adaugand un nod din H_2 vecin cu x , el se va deconecta la stergerea acestuia iar proprietatea de b-subgraf nu mai este indeplinita.



In mod analog si pentru H_2 . Astfel demonstrand ca un punct de articulatie x din G , se afla la intersectia a doua b-subgrafuri.

Facand cele doua demonstratii putem trage concluzia ca un punct de articulatie se afla la intersectia a cel putin doua b-subgrafuri si intersectia a cel putin doua b-subgrafuri este un punct de articulatie. Q.E.D.

b. Stim de la subpunctul a ca orice punct de articulatie in G se afla la intersectia a cel putin 2 b-subgrafuri. Astfel multimea nodurilor care se afla la intersectia a cel putin doua b-subgrafuri din G este chiar multimea $\mathcal{V}_2$.

i. $\mathcal{G}$ conex: Multimea $\mathcal{V}_2$ este formata din intersectia nodurilor ce se afla la intersectia b-subgrafurilor din $\mathcal{V}_1$ (subpunctul a). Cum graful G este conex, atunci stim ca exista un drum intre doua noduri care sunt puncte de articulatie (adica din multimea $\mathcal{V}_2$). Daca $\mathcal{G}$ nu ar fi conex atunci nu va exista un drum de la doua noduri oarecare din $\mathcal{V}_2$. Acest lucru inseamna ca nici G nu ar fi conex deoarece fiecare nod $x \in \mathcal{V}_2$ este conectat cu cel putin doua b-subgrafuri din $\mathcal{V}_1$. Ceea ce este contradictie cu proprietatea de conexitate a lui G . Astfel $\mathcal{G}$ este conex.

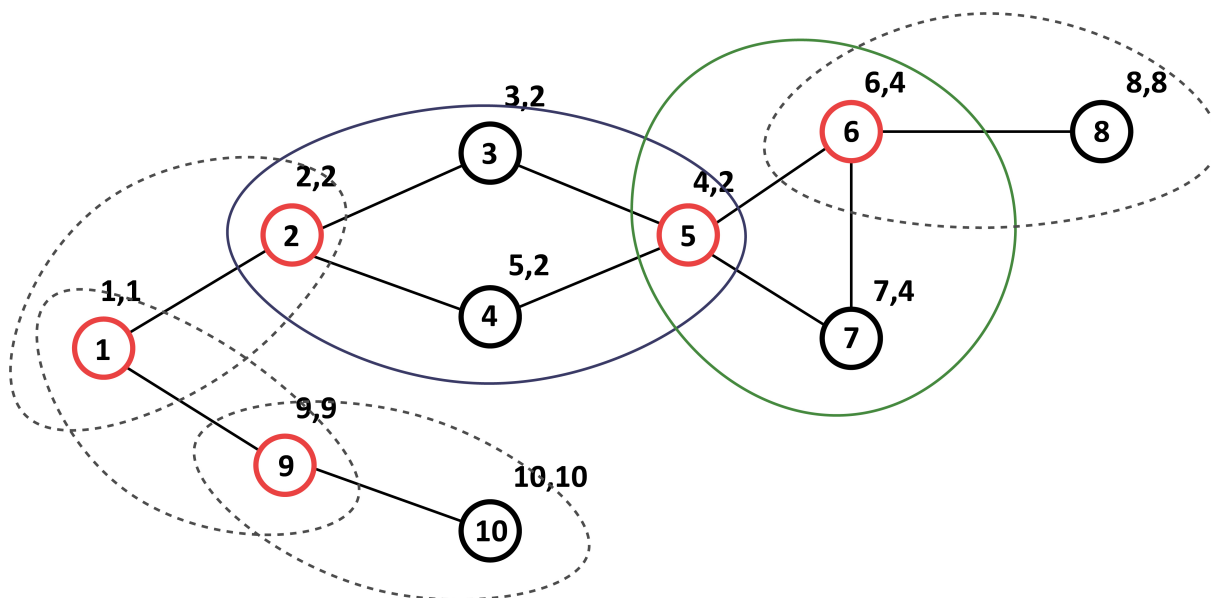
ii. $\mathcal{G}$ aciclic: Cunoscand faptul ca un b-subgraf este maximal, existenta unui ciclu in $\mathcal{G}$ ar determina existenta unui alt b-subgraf ce nu apartine lui $\mathcal{V}_1$ deoarece un ciclu formeaza un b-subgraf. Asta ar inseamna ca proprietatea de b-subgraf nu este maximala ceea ce este contradictie. Astfel am demonstrat ca $\mathcal{G}$ este aciclic.

Facand cele doua demonstratii i si ii rezulta ca $\mathcal{G}$ este un graf conex si aciclic adica un arbore. Q.E.D

c. Algoritmul este bazat pe o parcurgere DFS iar nodurile sunt vizitate recursiv de la nodul de start in ordinea data de nodurile nevizitate nodului curent.

Cum tabloul `order[]` nu se modifica decat la inceputul fiecarui apel recursiv, el va retine ordinea de vizitare DFS al oricarui nod.

Pentru tabloul `high[]` observam ca pentru un vecin care a fost deja vizitat ii schimbam valoarea cu $\min(\text{high}[x], \text{order}[v])$ iar la intoarcerea din recursie schimbam valoare lui `high[x]` in $\min(\text{high}[x], \text{high}[v])$. Putem observa ca pentru un nod al carui `high[x]` a fost schimbat, valoare lui se va propaga printre toti parintii pana vom ajunge la nodul care corespunde cu vecinul nodului de la care a inceput schimbarea lui `high[x]`. In acest moment stim sigur ca $\text{order}[x] \leq \text{high}[x]$ iar nodul x va fi afisat ca si punct de articulatie. Mai observam ca atunci cand un nod are ca vecin un nod deja vizitat ($\text{order}[x] \neq -1$) atunci putem deduce ca exista un ciclu intre cele doua noduri (cazul minimal al unui b-subgraf). Astfel, algoritmul marcheaza nodurile cu o valoare k_i care formeaza un circuit dar si acele noduri din circuit care formeaza alte circuite. Prin aceasta marcare recursiva vom ajunge la un b-subgraf deoarece eliminand un nod dintr-un circuit el va ramane conex. Un exemplu este dat in imaginea urmatoare:



- x, y corespunde cu valoare lui order respectiv high
- liniile continue sunt b-subgrafurile in care valoare lui high este aceeași cu excepție în unele cazurile în care două b-subgrafuri se intersectează
- liniile punctate corespund cu b-subgrafuri pe care algoritmul nu le "marchează" în mod direct, ele respectând celelalte condiții explicate mai jos
- nodurile cu roșu sunt afișate de algoritmul ca și puncte de articulație.

b-subgrafurile de ordin 2 nu sunt marcate direct dar pentru un nod c care are cel puțin doi vecini se și e punct de articulație se va respecta condiția $order[x] \leq high[v]$ (x și v sunt adiacente formând un b-subgraf). Astfel fiecare nod ce se afla la intersecția a două b-subgrafuri va fi afișat ca și punct de articulație. Este ușor de văzut pentru nodul rădăcină (primul nod vizitat) ce are cel puțin doi copii și nu există un circuit dat de cele 3 noduri (rădăcină și cei doi copii), el va fi punct de articulație și va fi afișat corect de algoritmul. Astfel algoritmul de mai sus este corect: el afișează toate punctele de articulație.

Problema 4

Demonstratie ajutatoare pentru b si c: Demonstram prin inductie ca după terminarea fiecărei bucle for T va fi format din arbori parțiali de cost minim.

Verificare: pentru $p = 2$ avem 2 noduri care vor fi conectate cu singura muchie din graf. După terminarea buclei for T va fi compus dintr-un singur arbore parțial de cost minim: graful inițial.

Presupunem ca T neconex și $T = \{T_1, T_2, \dots, T_p\}$, unde $T_i, i = 1, p$ este arbore parțial de cost minim în subgraful indus de $V(T_i)$ în G și demonstrăm ca după terminarea buclei for $T = \{T_{1'}, T_{2'}, \dots, T_{p'}\}$ unde $T_i, i = 1, p'$ este arbore parțial de cost minim în subgraful indus de $V(T_i)$ în G . Fie e muchia de cost minim cu capetele în doi arbori parțiali de cost minim din T : T_i respectiv T_j $i \neq j, i = 1, p, j = 1, p$. Unind cei doi arbori vom obține un arbore T_k și este de cost minim în subgraful indus de $V(T_k)$ în G deoarece muchia adăugată are costul minim în G și unește alți 2 arbori parțiali de cost minim. Acest arbore este unic deoarece toate muchiile au costuri diferite. T se va modifica astfel $T = T \setminus \{T_i, T_j\} \cup T_k$, iar proprietatea lui T de a avea ca și componente arbori parțiali de cost minim este păstrată. Repetând procesul, vom ieși din bucla for iar $T = \{T_{1'}, T_{2'}, \dots, T_{p'}\}$ unde $T_i, i = 1, p'$ arbore parțial de cost minim, ceea ce trebuia demonstrat.

- Adăugând câte o muchie e ($e = uv : uv \in E, u \in V(T_i), v \notin V(T_i)$) care conectează două

componente conexe deiferite se obtine o noua componenta conexa. In cazul cel mai nefavorabil se conecteaza doua componente care nu au fost conectate la pasii precedenti din bucla for, astfel numarul de componente conexe rezultat dupa executia buclei for va fi jumătate (mai exact $\lceil p/2 \rceil$, unde p este numarul de componente conexe inainte de executia buclei for).

b. Initial exista $p=n$ componente conexe (varfurile din graful G), acestea fiind aciclice. Deoarece o muchie uv conecteaza doar componente conexe diferite ($u \in V(T_i), v \notin V(T_i)$), oricare doua componente vor fi conectate prin exact o muchie, astfel noile componente conexe rezultate vor fi aciclice. Stim acest lucru din demonstratia ajutatoare de mai sus. In concluzie, la terminarea executiei buclei while, T va fi conex iar fiind format din componenete conexe aciclice conectate prin exact o muchie, va fi la randul lui aciclic.

c. La fiecare pas din bucla for se alege muchia de cost minim care conecteaza doua componente conexe diferite (arbori partiali de cost minim, cunoscand acest lucru din demonstratia de mai sus). Astfel alegand la fiecare pas muchia de cost minim, graful rezultat fiind aciclic si functia de cost fiind injectiva T rezulta un arbore partial de cost minim.

d. Din $a. \Rightarrow$ ca dupa fiecare iteratie while numarul de componente conexe se injumatateste (cel putin). Bucla se opreste cand se obtine un graf conex deci numarul de iteratii while este $\mathcal{O}(\log_2 n) \subset \mathcal{O}(\log n)$.

La fiecare iteratie while algoritmul cauta muchia de cost minim astfel in cel mai nefavorabil caz se va parcurge (in bucla for) toata multimea de muchii E ($|E| = m$), astfel complexitatea fiecarei iteratie while este $\mathcal{O}(m)$.

In concluzie complexitatea finala a algoritmului este $\mathcal{O}(m \log n)$.